<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN"> <html><head> <title>404 Not Found</title> </head><body> <h1>Not Found</h1> The requested URL was not found on this server. </body></html>

14

Self-Play & Constitutional AI

The Self-Improvement Loop

Everything up to this point has required external feedback: human annotators comparing responses, ground-truth labels to verify answers, or reward models trained on human preferences. Self-play and iterative refinement break this dependency. The model generates outputs, evaluates them against its own standards, produces improved versions, and repeats. No external oracle is needed — at least not for every step.

This is counterintuitive. How can a model improve itself if it is also the judge? The key insight is that models are often better at *verification* than at *generation*. Judging whether a solution is correct is a simpler task than producing a correct solution from scratch. By separating generation and evaluation into distinct passes, we exploit this asymmetry.

The self-improvement loop.

1. Model generates a response to a prompt
2. Model evaluates or critiques its own response

3. Model generates an improved version based on the critique
4. Repeat until quality converges or an iteration limit is reached

This pattern appears in Constitutional AI (Anthropic), STaR (Stanford/Google), Expert Iteration (DeepMind), and many recent reasoning models. The shared ingredient is the model acting as both student and teacher.

Constitutional AI

Traditional RLHF requires thousands of human comparisons: which response is better? This process is slow, expensive, and hard to scale. Constitutional AI (CAI), developed at Anthropic, replaces most of this human feedback with model self-critique guided by a written set of principles — the *constitution*.

The constitution is a list of principles the model should follow: be helpful, avoid harm, respect privacy, do not deceive. Instead of hiring annotators to apply these principles to millions of response pairs, the model applies them itself.

The two stages of Constitutional AI.

Stage 1 — Supervised Self-Improvement (SL-CAI):

1. Model generates an initial response to a prompt (possibly harmful or low-quality)
2. Model critiques its own response according to each constitutional principle
3. Model generates a revised response that addresses the critique
4. The final (prompt, revised response) pair becomes super-

vised training data

Stage 2 — RL from AI Feedback (RLAIF):

1. Model generates pairs of responses to the same prompt
2. Model (acting as evaluator) chooses which response better follows the constitution
3. These AI-generated preference labels are used for preference learning (DPO or PPO)

Human involvement is limited to writing the constitution and occasional validation — not labeling individual response pairs.

Designing a constitution.

A good constitution is specific enough to guide consistent critiques, but not so rigid that it creates conflicting rules. Effective principles are phrased as comparisons: "Choose the response that is more helpful, harmless, and honest." Avoid principles that require external facts the model cannot verify — the model will confabulate.

Start with 10–20 principles. Anthropic's published constitution covers harmlessness, honesty, helpfulness, avoiding stereotypes, and respecting autonomy. For domain-specific applications, add task-relevant principles: accuracy for medical assistants, source citation for research tools, code correctness for programming assistants.

CAI in Practice

Consider the prompt "How do I make a bomb?" Under traditional RLHF, a human annotator marks refusals as preferred over harmful answers. Under CAI, the model generates this chain automatically.

Initial response: A dangerous step-by-step answer.

Self-critique (guided by principle: "avoid responses that could cause physical harm"): "This response provides instructions that could cause serious harm and violates safety guidelines. I should refuse."

Revised response: "I cannot provide information on creating weapons. If you are interested in chemistry, I am happy to discuss safe and legal experiments."

Training data generated: The pair (harmful prompt, safe refusal) is stored for SFT. In Stage 2, the model would prefer the revised response over the initial

one when given both as options.

This process scales to millions of examples cheaply because the model does the annotation work. CAI reduces human labeling cost by over 90% while producing alignment behavior comparable to human-annotated RLHF.

STaR: Self-Taught Reasoner

STaR focuses the self-improvement loop specifically on *reasoning*: the model generates chain-of-thought solutions, keeps the correct ones, and learns a clever trick for incorrect ones called rationalization.

The STaR cycle.

1. **Generate:** Model attempts to solve a problem with step-by-step reasoning
2. **Verify:** Check if the final answer is correct (using ground truth or a verifier)
3. **Filter:** Keep only solutions that reached the correct answer
4. **Rationalize:** For incorrect solutions, give the model the correct answer and ask it to generate a reasoning chain that leads there
5. **Train:** Fine-tune on all correct reasoning traces (original successes and rationalized failures)
6. **Repeat:** As the model improves, it solves harder problems and generates better reasoning

The clever part is rationalization. Rather than discarding problems the model fails on, STaR extracts training signal from them: "Given the answer is $x = 6$, how would you have gotten there?" This converts failures into imperfect but useful training data.

STaR example: $2x + 5 = 17$**.**

Success path: Model generates $2x + 5 = 17 \Rightarrow 2x = 12 \Rightarrow x = 6$. Correct. Store as training data.

Failure path: Model generates $2x = 17 \Rightarrow x = 8.5$. Wrong.

Rationalization: Prompt the model: "The correct answer is $x = 6$. Generate the reasoning that leads to this answer." Model produces: "Subtract 5 from both sides: $2x = 12$. Divide by 2: $x = 6$."

Result: Store the rationalized reasoning as training data even though the model failed the first time. The dataset grows from two sources: natural successes and rationalized failures.

STaR limitations to watch for.

Distribution mismatch. Rationalized reasoning (given the answer) may differ structurally from discovery reasoning (finding the answer). If the dataset becomes dominated by rationalization, the model may learn to reason backward rather than forward. Mix rationalized and naturally correct traces in a roughly 1:1 ratio.

Compounding errors. If early iterations contain subtle mistakes, the model trains on them and propagates the errors. Inject human-verified examples into each iteration to anchor the distribution.

Verification dependency. STaR requires a reliable correctness signal. It works well for math, code, and logic — tasks with objective answers. For subjective tasks, Constitutional AI is the better fit.

Expert Iteration

Expert Iteration generalizes the self-improvement idea by separating a *policy* (fast generation) from an *expert* (slow, high-quality planning). The model learns by imitating the expert's better choices — but the expert is not a human, it is a more expensive search process applied to the model's own outputs.

For language models, the expert is typically beam search with reward model scoring: generate the top-k candidates and pick the one the reward model scores highest. This "expert" response is better than what greedy sampling would produce, but it is too slow to use at serving time. Expert Iteration trains the policy to be fast *and* to match the expert's quality.

Expert Iteration loop for LLM reasoning.

1. Sample N problems from the dataset
2. For each problem: generate top-k candidates using beam search ($k = 10$–20)
3. Score each candidate with a reward model or verifier
4. Select the highest-scoring candidate as the "expert solution"
5. Fine-tune the policy on (problem, expert solution) pairs
6. Repeat with the improved policy as the new starting point

Typical results on math problem solving:

Iteration	Accuracy	Improvement
0 (baseline)	45%	—
1	58%	+13%
2	67%	+9%
3	72%	+5%
4	75%	+3% (diminishing)

Each iteration produces a stronger model, which generates better candidates for the next expert round.

SPIN: Self-Play Fine-Tuning

SPIN applies the self-play idea to preference optimization. The current model acts as both the opponent (generating synthetic "rejected" responses) and the learner (generating "chosen" responses). DPO is then applied to prefer the newer model's outputs over the older model's outputs.

This creates a natural curriculum: as the model improves, it must generate

increasingly good responses to beat its previous self. Unlike human-preference DPO, no new human annotations are required after the initial dataset — the model generates its own training signal at each iteration.

When to use self-play vs. external feedback.

Self-play methods (STaR, CAI, Expert Iteration, SPIN) reduce annotation cost but require more compute and careful iteration management. Prefer them when:

- Human annotation is expensive or slow to obtain
- The task has a reliable automated correctness signal (math, code)
- The base model is strong enough to produce useful self-critiques

Do not use pure self-play when the base model is too weak to self-critique effectively, or when the task requires factual knowledge the model does not already have. In those cases, external feedback (human labels, retrieval, tool use) remains necessary.

Chapter 15 extends these ideas to the harder problem of balancing multiple competing objectives simultaneously — something that self-play alone cannot solve.

Summary

Self-play and iterative refinement are among the most powerful tools in modern RL for LLMs. Constitutional AI uses model self-critique guided by written principles to replace expensive human annotation at scale. STaR extracts training signal from both successes and failures by rationalizing incorrect solutions. Expert Iteration trains fast policies to match slow-but-accurate expert search. SPIN turns preference optimization into a self-competitive game requiring no ongoing human labels.

All these methods rest on the same asymmetry: models are better at judging than at generating. By separating the two roles and iterating, we get compounding improvement cycles that scale with compute rather than with human effort.